

VORBEREITUNG ZUR VORLESUNG

Handout 03 – Eigene Images bauen und konfigurieren

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Dockerfile, Layer, Cache, Umgebungsvariablen
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Vom fremden zum eigenen Image

Bisher haben Sie fertige Images benutzt (`python` , `ubuntu`). Jetzt bauen Sie ein **eigenes** Image, das Ihren Code bereits enthält. Die Bauanleitung steht in einer Textdatei namens `Dockerfile` .

Das Dockerfile

Jede Zeile ist eine Anweisung, und (fast) jede Anweisung erzeugt eine neue **Schicht** (Layer):

```
FROM python:3.13-slim      # Basis-Image, worauf wir aufsetzen
WORKDIR /app               # Arbeitsverzeichnis im Image
COPY log_tool.py .         # Datei vom Build-Kontext ins Image kopieren
ENV LOG_FILE=/data/log.txt # Voreinstellung als Umgebungsvariable
ENTRYPOINT ["python", "log_tool.py"] # was beim Start ausgeführt wird
```

- **FROM** wählt die Grundlage. Wir nehmen ein schlankes Python-Image (`-slim`).
- **COPY** bringt Dateien aus dem *Build-Kontext* (dem Ordner, den Sie beim `docker build` . angeben) ins Image.
- **ENTRYPOINT** legt den festen Startbefehl fest; die Argumente `add` / `list` hängen Sie beim `docker run` an.

Layer und Cache – warum die Reihenfolge zählt

Docker speichert jede gebaute Schicht zwischen. Ändert sich eine Schicht, müssen **alle darunterliegenden neu gebaut** werden, die darüber bleiben aus dem Cache. Deshalb ordnet man ein Dockerfile so, dass sich selten Änderndes **oben** steht und oft geänderter Code **unten**:

```
COPY requirements.txt .      # ändert sich selten
RUN pip install -r requirements.txt
COPY log_tool.py .          # ändert sich oft -> steht bewusst weiter unten
```

So muss `pip install` nicht bei jeder Code-Änderung erneut laufen. Diese Reihenfolge sehen Sie ab Übung 04.

Merke

Ein Neubau nach kleiner Code-Änderung sollte Sekunden dauern, nicht Minuten. Wenn nicht, steht vermutlich etwas Langsames (Installation) zu weit unten im Dockerfile.

Der Build-Kontext und `.dockerignore`

Beim `docker build .` schickt Docker den **gesamten Ordner** an den Daemon – das ist der Build-Kontext. Eine `.dockerignore` (wie in diesem Ordner) schließt Unnötiges aus (`__pycache__`, PDFs, ...), damit der Build schlank und schnell bleibt und keine Geheimnisse versehentlich ins Image wandern.

Konfiguration per Umgebungsvariable

Guter Container-Code liest seine Einstellungen aus **Umgebungsvariablen**, statt sie fest zu verdrahten. So läuft *dasselbe* Image in verschiedenen Situationen:

```
docker run -e LOG_FILE=/data/andere.txt log-tool:1.0 ...
```

`ENV` im Dockerfile liefert die Voreinstellung, `-e` beim Start überschreibt sie. Dieses Prinzip – Konfiguration von außen – zieht sich durch alle weiteren Übungen (Datenbank-Host, Benutzer, Passwort).

Tags: Versionen für Images

`docker build -t log-tool:1.0 .` gibt dem Image einen Namen und eine Version (`1.0`). Bauen Sie später `log-tool:1.1`, existieren beide nebeneinander. Ohne ausdrücklichen Tag vergibt Docker `latest` – was in echten Projekten gefährlich ist, weil „latest“ ein bewegliches Ziel ist.